Ganak: Engineering an Exact Model Counter

Mate Soos
Argot Collective

joint work with Kuldeep S. Meel

Ganak (IJCAI'19): Sharma, Roy, Soos, Meel Ganak2 (CAV'25): Soos, Meel

IIT Bombay, 10th of November 2026

Outline

1. Model counting and Ganak
2. Preprocessing
3. Search
4. Fields and results

Propositional model counting

Counting (#SAT)

How many solutions does a CNF have?
$(x_1 \vee x_2) \wedge (\neg x_1 \vee x_3)$ has 4.

One solution: NP. Counting them: **#P-complete**.

This talk: *exact*. Approximate, with guarantees: ApproxMC.

Weighted counting

Each literal has a weight; a solution's weight is the **product** of its literals' weights; the count is the **sum** over the solutions. Weights: rationals, complex numbers, polynomials...

Projected counting

Count only distinct assignments to a *projection set* P:

P	other vars
0001	010111
0001	101010
1011	111010

3 solutions, but only **2** over P.

Uses: probabilistic inference, quantitative verification (information leakage), reliability, product-line configuration, quantum circuit simulation.

ApproxMC: Chakraborty, Meel, Vardi, CP'13.

Model Counting Competition, 2024–2026

Track (exact)	2024	2025	2026
1: Model counting	**1st**	**1st**	2nd
1F: MC, 120 s	—	—	**1st**
2: Weighted MC	**1st**	—	**1st**
3: Projected MC	**1st**	**1st**	**1st**
4: Proj. weighted MC	**1st**	**1st**	**1st**
5B: Complex weights	—	—	**1st**

—: no such track/award.
2026 MC: d4 63, Ganak 62 solved.

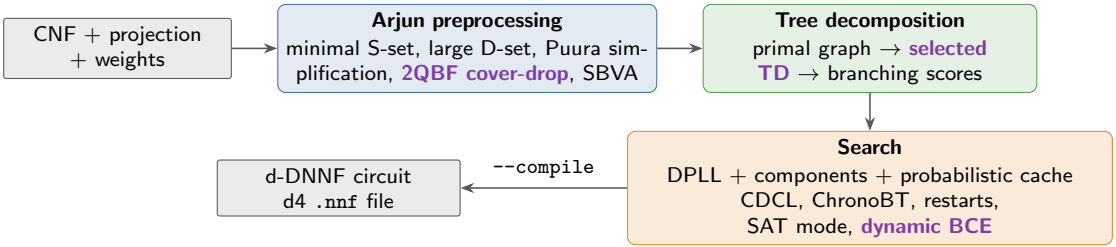
FLoC 2026 medal

Most Competitive Solver over the Last 6 Years

Ganak: Mate Soos, Jiong Yang, Kuldeep S. Meel, Shubham Sharma

Source: `https://mccompetition.org`

Ganak at a glance



The search trace *is* a d-DNNF[1]: decision nodes are **OR** nodes, component splits are **AND** nodes. Ganak can write it out in the d4 `.nnf` format[2] for repeated queries, enumeration, or synthesis.

Sum of products: decisions **add**, components **multiply**, so any field works.

[1] Darwiche & Marquis, *A Knowledge Compilation Map*, JAIR 2002; Huang & Darwiche, *DPLL with a Trace*, IJCAI 2005

[2] Lagniez & Marquis, *An Improved Decision-DNNF Compiler*, IJCAI 2017

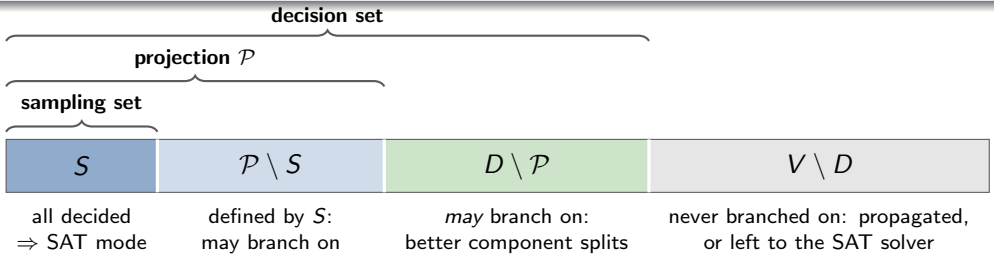
Dual independent set

Definition (Dual independent set)

For a formula F over V with projection set $\mathcal{P}$:

1. **S-set** (sampling set) $S \subseteq \mathcal{P}$ *defines* $\mathcal{P}$: all of S decided $\Rightarrow$ SAT mode
2. **D-set** (decision set) $D \supseteq \mathcal{P}$ is *defined by* $\mathcal{P}$: safe to branch on

Traditionally $D = S \subseteq \mathcal{P}$. We aim for $S \subseteq \mathcal{P} \subseteq D$: a *small* S to reach SAT mode early, a *large* D to split into components well (CAV'25).



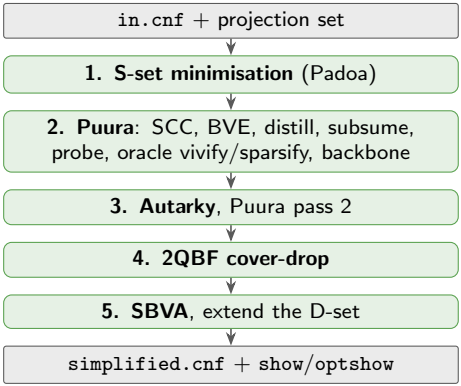
Preprocessing with Arjun

Small S-set + large D-set

Most variables are *defined* by others (Padoa's theorem: two renamed copies cannot differ if the projection agrees). Arjun shrinks the projection to a small **S-set**, and grows it to a large **D-set** the counter *may* branch on: gate outputs whose inputs are all in D, then Padoa checks.

Simplified CNF

Fewer variables, fewer and shorter clauses. Every step must be **count-preserving** for the projection. BVE and equivalent-literal merging do most of the structural work.



Arjun: Soos & Meel, ICCAD'22; preprocessor: Soos & Meel, DAC'24. Definability for counting: Lagniez, Lonca, Marquis, IJCAI'16. Minimal independent support: Ivrii, Malik, Meel, Vardi, Constraints'16.
Puura reuses SharpSAT-TD's preprocessor (Korhonen & Järvisalo, CP'21). SBVA: Haberlandt, Green, Heule, SAT'23.

The workhorse: bounded variable elimination (BVE)

To eliminate x:

- Replace every clause with x and every clause with $\neg x$ by **all their resolvents** on x
- Drop the tautologies
- Only do it if the formula does not grow: **bounded**

The result is $\exists x.\, F$: the count projected away from x is unchanged. So only variables **outside the projection** are eliminated

	$\neg x \vee d$	$\neg x \vee \neg a \vee e$
$x \vee a \vee b$	$a \vee b \vee d$	$a \vee \neg a \vee \ldots$: taut.
$x \vee c$	$c \vee d$	$c \vee \neg a \vee e$

4 clauses $\rightarrow$ **3 resolvents**, none with x: fewer clauses, so eliminate x

Davis & Putnam, JACM 1960. BVE: Eén & Biere, SAT'05.

Arjun by the numbers

`mc2023_track3_079`

stage	variables	clauses
input CNF	26 351	112 951
S-set (projection)	75 → **12**	—
Puura pass 1	414	2 572
+ autarky, pass 2	413	2 568
SBVA	420	**2 504**
D-set extension	17 → **49**	—

98% of variables and clauses gone, in 8.5 s

1600 MCC'23+'24 CNFs

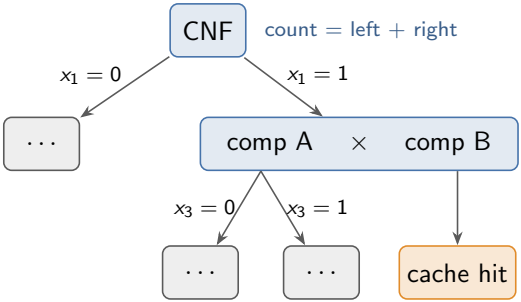
- Typically ∼87% of variables and ∼67% of clauses removed, count unchanged
- Structure: **BVE** and equivalent-literal merging do nearly all of it
- Time: backbone (CadiBack) 45%, oracle vivify/sparsify 34%, BVE only 8%

Everything else exists to set BVE up to do its job.

BVE: Eén & Biere, SAT'05. CadiBack: Biere, Froleyks, Wang, SAT'23. Preprocessing for counting: Lagniez & Marquis, AAAI'14.

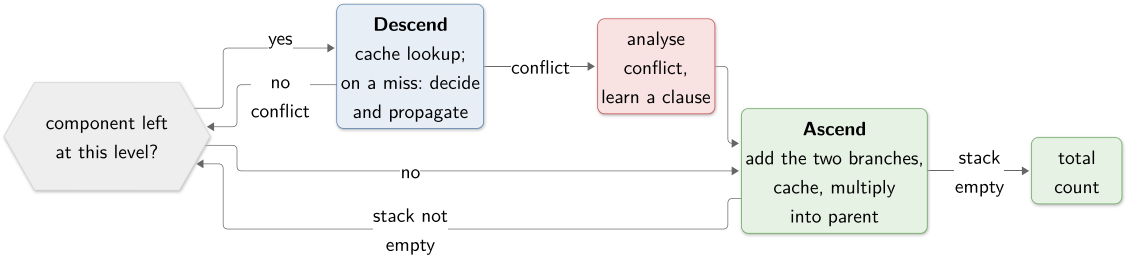
The search: branching and component decomposition

- Branch on a variable: the count is the **sum** of the two branches
- The residual formula falls apart into variable-disjoint **components**: the count is their **product**
- Count each component once, then **cache** it: the same component shows up again and again
- Conflicts: learn clauses, as in CDCL



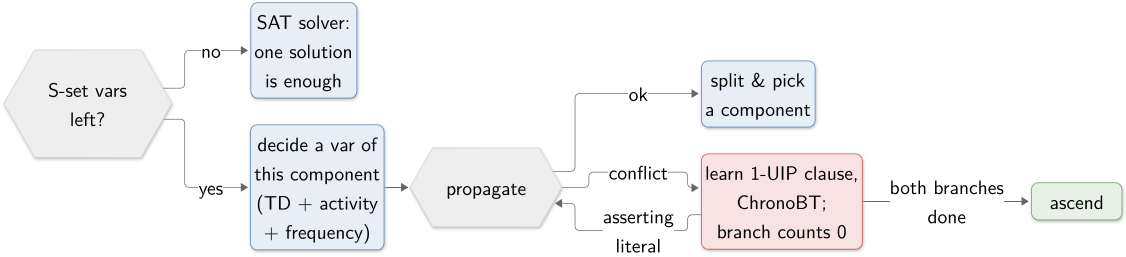
Components: Bayardo & Pehoushek, AAAI'00. Component caching: Bacchus, Dalmao, Pitassi, FOCS'03; Sang et al. (Cachet), SAT'04; Thurley (sharpSAT), SAT'06.

The main loop



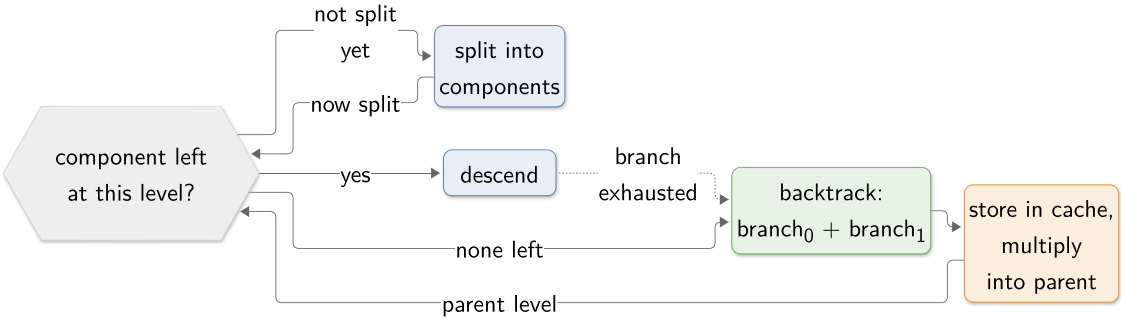
- **Descend**: look the component up in the cache; on a miss, decide a variable in it and propagate. The next iteration cuts the result into components
- **Ascend**: add and multiply the sub-counts back up the stack
- We count on the way *up*: much cheaper, e.g. a conflict means there is nothing to count

Descending: decide, propagate, analyse conflicts



- This is the CDCL part, very close to a SAT solver
- Counting-specific: branch only on variables of the *current component*, and a conflict sets the count of its branch to 0

Ascending: components, cache, count arithmetic



- Independent components **multiply**, the two values of a decision variable **add**
- Every component count is cached, so it is not searched again until evicted
- The same $+$ and $\times$ for every field: the engine never knows which one it counts in

Probabilistic component caching

- A component = its unassigned variables + its unsatisfied clauses. Storing it exactly is **large**: hundreds of variables and clause IDs.
- Ganak stores only a **128-bit hash** of it, seeded from `--seed`
- Smaller entries ⇒ many more components cached, and faster lookups
- The price: a collision gives a wrong count, with probability $\approx n^2/2^{128}$ over n stored components, if the hash behaves like a random function

Probabilistically exact

The count is exact unless a hash collision happened. Rerun with another `--seed` for an independent check.

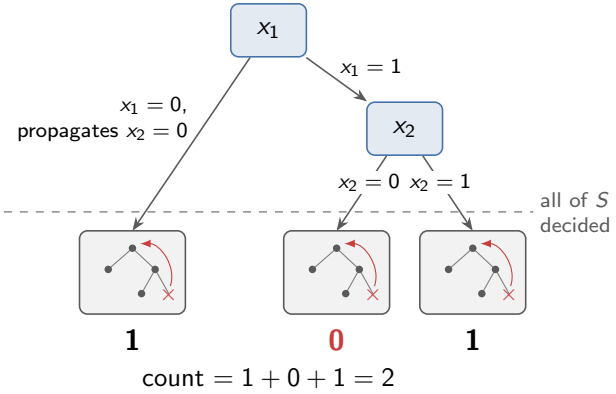
Introduced in Ganak (Sharma, Roy, Soos, Meel, IJCAI'19) with a universal hash family and a formal bound; today MurmurHash3, 128 bits. Entries are evicted by last use once `--maxcache` is reached.

S-set and calling a SAT solver

Once all variables of the **S-set** S (an independent support of the projection) are decided, there is **at most one** solution over $\mathcal{P}$ left

So call a **SAT solver** on the residual formula: it answers 1 or 0. No more branching, components or cache below that point

$S = \{x_1, x_2\}$. Exploited before by gpmc.



gpmc: Suzuki, Hashimoto, Sakai, 2017.

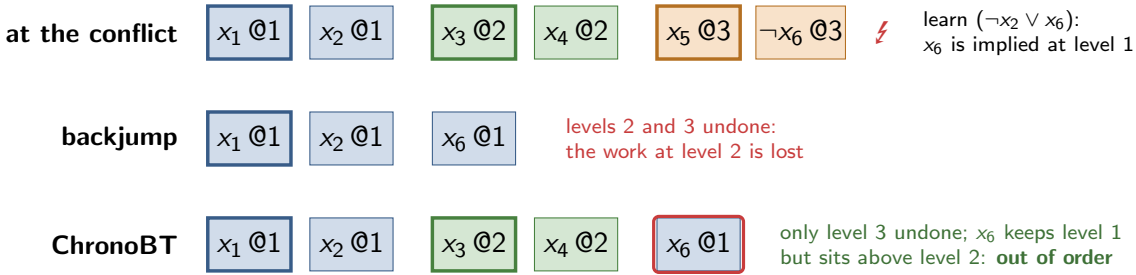
Enhanced residual formula processing

Lift the SAT solver's techniques into the counter's SAT mode:

- **VSIDS scoring**: counters branch with VSADS[3], which is good for splitting into components. For finding *one* solution, VSIDS is cheaper to compute and better.
- **Restarts** (Luby[4]): we only need one solution, so restarting the SAT search is safe.
- **Polarity caching**[5]: remember the last polarity, so restarts do not throw the work away.

[3] Sang et al., 2005
[4] Luby et al., 1993
[5] Pipatsrisawat & Darwiche, 2007

Chronological backtracking (ChronoBT)

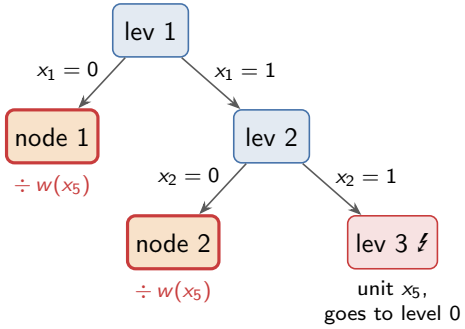


- **Backjumping throws away counted solutions**, so counters backtrack one level, and drop the learnt clause if it would break the propagation invariants
- **ChronoBT**[6] relaxes the invariants: implication levels may be **out of order** on the trail
- **Keep** the learnt clause, and force the literal it implies, **without** undoing the levels in between

[6] Nadel & Ryvchin, *Chronological Backtracking*, SAT 2018. In counting: Möhle & Biere, GCAI 2019

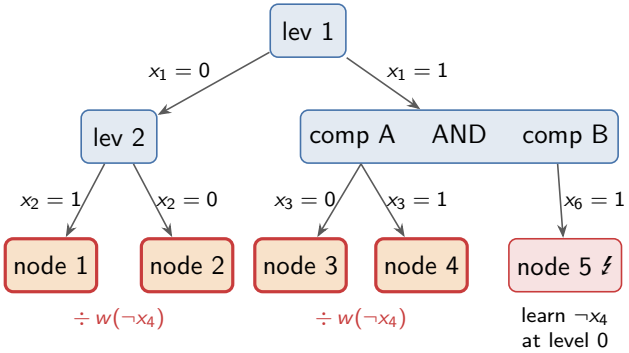
ChronoBT needs compensating weights

1. Nodes 1 and 2 are counted. x_5 holds in all their models, so $w(x_5)$ is already in their counts
2. At level 3: conflict, learn the **unit** x_5. ChronoBT undoes only level 3, and puts x_5 at **level 0**, above every decision
3. Level-0 weights multiply the **whole** count at the end: $w(x_5)$ would be counted twice in nodes 1 and 2
4. So **divide** $w(x_5)$ out of every finished branch on the path: nodes 1 and 2



ChronoBT needs compensating across components, too

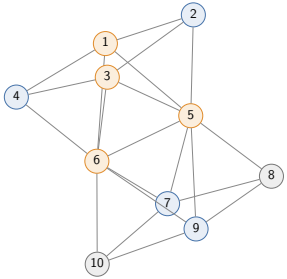
1. x_4 is in the components of nodes 1–4, but not 5. Yet at 5 we learn $\neg x_4$ at level 0
2. How? Components are computed from the *irredundant* clauses only. Learnt clauses can **connect** components
3. So conflicts can be over variables *outside* the component examined
4. So divide $w(\neg x_4)$ out of the finished branch $x_1 = 0$ (nodes 1+2) *and* the finished sibling component (nodes 3+4)



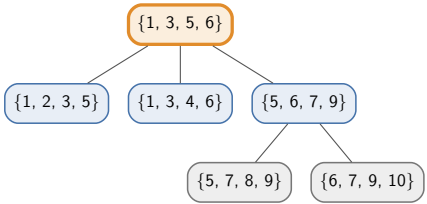
Tree decomposition: an example

Build a tree decomposition (TD) of the primal graph, with FlowCutter as in SharpSAT-TD[7]:

```
 1  2  5
-2  3  5
 3  4  6
-1 -4  6
 7  8  5
-8  9  5
 9 10  6
-7 -10 6
 5 -6
```



primal graph: an edge per pair of vars in a clause



tree decomposition, width 3

- Every clause fits in some bag; the bags holding a variable form a subtree. Width = largest bag − 1
- The centroid bag $\{1, 3, 5, 6\}$ splits the tree into balanced parts. Decide its variables first: 2, 4 and $\{7, 8, 9, 10\}$ end up in different **components**

[7] FlowCutter: Hamann & Strasser, JEA 2018; Strasser, PACE 2017. SharpSAT-TD: Korhonen & Järvisalo, CP'21

Tree decomposition guides branching

TD score $td(v)$: the closer v's bag is to the centroid, the higher. In the example: centroid 100, one bag away 50, two away 0. It is mixed into the branching score:

$$score(v) = \frac{act(v)}{3} + w_{TD} \cdot td(v) + \frac{freq(v)}{25}$$

Recent changes:

- **TD selection**: local search with a tick-based (i.e. deterministic) timeout; the narrowest TD wins
- **Flat rule**: if width $\geq 50\%$ of the nodes, the graph is too dense for the TD to say anything: $w_{TD} = 0$

1600 MCC'23+'24 instances

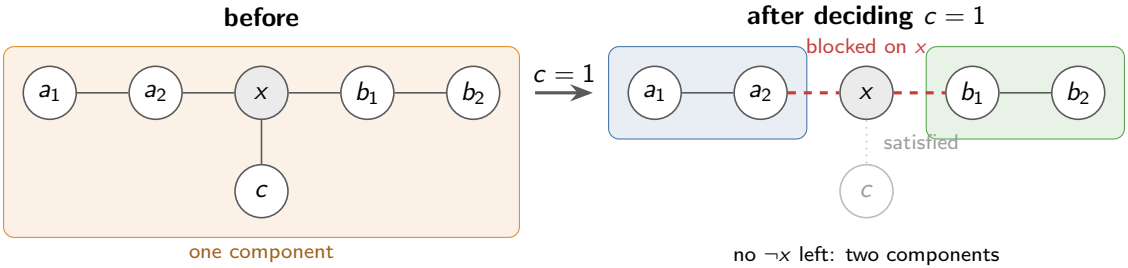
- selection: 1181 → 1198
- + flat rule: → **1205** (+37, −13)
- PAR2 2084 → 1950
- commonly solved (1168): 285 ks → **227 ks** (−20%)

Width alone does not predict the work: TDs of the same width can differ 2.4× in decisions.

Dynamic blocked clause elimination

- C is **blocked** on $l \in C$ if every resolvent of C on l is a tautology. Removing C keeps the count projected away from $\mathrm{var}(l)$
- **Dynamic**[8]: under the current assignment satisfied clauses vanish, so more clauses become blocked. Ignore them when computing **components**

$$(a_1 \vee a_2) \wedge (\neg a_2 \vee x) \wedge (x \vee b_1) \wedge (b_1 \vee b_2) \wedge (\neg x \vee c), \quad x \text{ outside the D-set}$$



[8] Lagniez, Marquis, Biere, SAT'24. Blocked clauses: Kullmann, 1999

Dynamic BCE in Ganak

- Only on variables we never branch on: outside the D-set, as removal is count-preserving only when $\mathrm{var}(l)$ is projected away
- Finer splits $\Rightarrow$ smaller components $\Rightarrow$ more cache hits
- Incremental, and undone in groups when backtracking
- ChronoBT can unassign a variable while later ones stay set, so those clauses are re-processed

1600 MCC'23+'24 instances

- **+11** solved, −1 lost: 1210 → **1220**
- PMC 282→288, PWMC 336→341
- PAR2 1951 → 1899
- projected, >10 s: decisions ×0.42, time ×0.87 (geo. mean)
- 25 instances ≥2× faster, none ≥2× slower

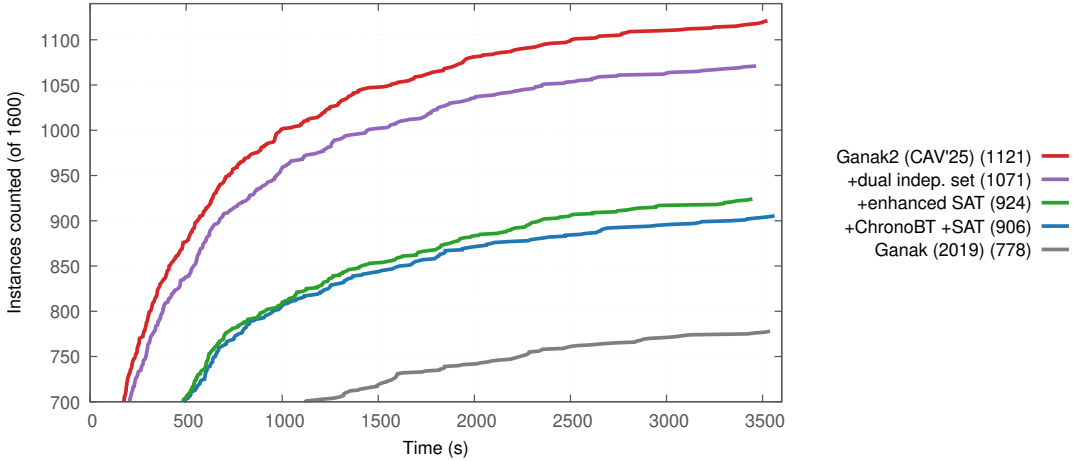
Counting over any field

- Weighted counting is a sum of products: a **semiring** would do
- But compensating weights must **divide**: Ganak needs exact division by the weights, e.g. a **field**. Unweighted counts never divide, so integers are fine
- The counter is written against an abstract field interface: adding a new field takes ≈10 minutes
- 2026 brought a new competition track with **complex** weights (AMC)

`--mode`	field
0	integers (default)
1	rationals (GMP)
2	complex rationals
3	multivariate polynomials over $\mathbb{Q}$
4	parity (mod 2)
5	integers mod a prime
6	complex floats (MPFR)
7	real floats (MPFR)
13	Laurent polynomials over $\mathbb{Q}$

Algebraic model counting: Kimmig, Van den Broeck, De Raedt, J. Applied Logic 2017.

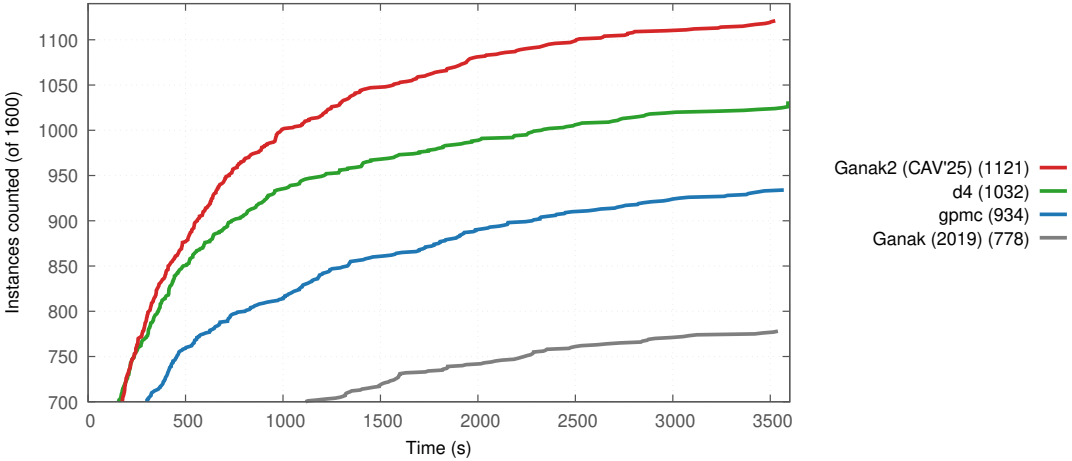
Results at CAV'25: from Ganak to Ganak2



1600 MCC 2023+2024 instances, 3600 s, CAV'25 cluster.

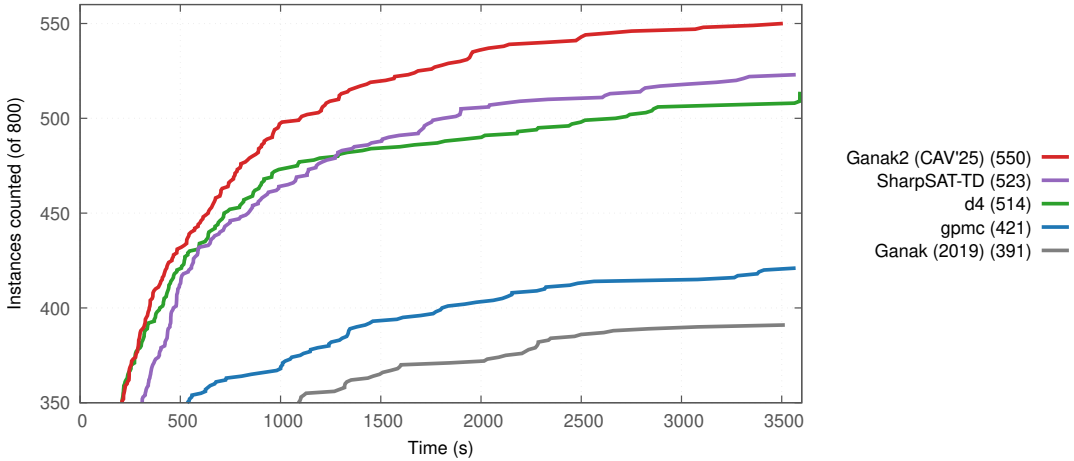
Soos & Meel, *Engineering an Efficient Probabilistic Exact Model Counter*, CAV'25.

Results at CAV'25: against other counters



1600 MCC 2023+2024 instances, 3600 s, CAV'25 cluster.

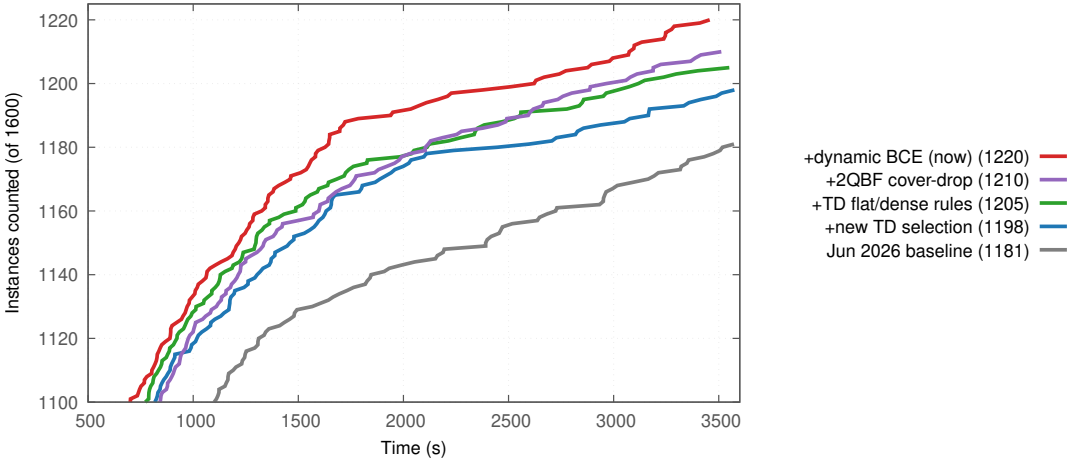
Results at CAV'25: unprojected counting



The 800 MC + WMC instances (tracks 1–2), 3600 s, CAV'25 cluster. SharpSAT-TD does not support projection.

SharpSAT-TD: Korhonen & Järvisalo, CP'21.

Results since: TD, 2QBF, BCE



Same 1600 instances, 3600 s, but on a **different cluster** (Trillium) and with a June 2026 baseline: not comparable with the CAV'25 graphs. Each step is a full run of the then-current build.

Conclusions

Ganak is an exact model counter built from many parts working together:

- **Preprocessing**: Arjun's minimal S-set, large D-set and simplification, now with **2QBF cover-drop**
- **Probabilistic component caching**
- **Dual independent set** (S-set, D-set) with a CDCL SAT solver, ChronoBT and compensating weights, now with **dynamic BCE**
- **Tree decomposition**, **now selected and gated**
- **Any field**: integers, exact rationals, complex rationals, (Laurent) polynomials over $\mathbb{Q}$, integers mod a prime, parity, real and complex floats

Since June 2026: **1181** $\rightarrow$ **1220** of 1600 instances, PAR2 2084 $\rightarrow$ 1899.

Ganak: Sharma, Roy, Soos, Meel, IJCAI'19. Ganak2: Soos & Meel, *Engineering an Efficient Probabilistic Exact Model Counter*, CAV'25.

Code: 

Try it in your browser: `https://www.msoos.org/ganak/`

2QBF cover-drop: removing what cannot matter

$\psi(X)$: the clauses over the projection set X only;
$C_i(X, Y_i)$: components over *disjoint* non-projected Y_i.
We count $\#_X\big(\psi \wedge \bigwedge_i \exists Y_i.\, C_i\big)$.

For each C_i, solve the **2QBF**
$\forall X.\, \psi(X) \rightarrow \exists Y_i.\, C_i(X, Y_i)$

- **true**: drop C_i and its Y_i entirely
- **false**, but only a few X-cubes have no Y_i: **replace** C_i by clauses blocking those cubes
- weighted Y_i, or out of budget: keep

CEGAR over X; each counterexample is shortened to a cube by local search and SAT calls.

1600 MCC'23+'24 instances

- changes **255** instances (223 projected)
- removes 2.4 M clauses, 133 k vars
- **+7** solved, −0 lost (PMC 278→282, PWMC 333→336)
- PAR2 1969 → 1950

Cost: 61 ks of extra preprocessing over the set, so time on the commonly solved ones goes up 4%.